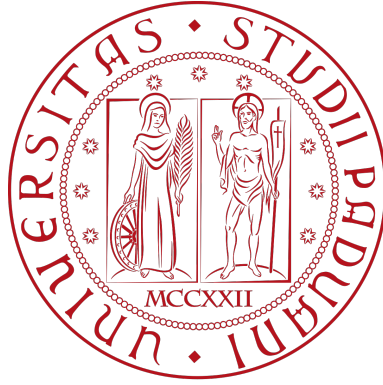




UNIVERSITÀ DEGLI STUDI DI PADOVA
Department of Mathematics “Tullio Levi-Civita”
Master’s Degree in Computer Science
Academic year 2025/2026
Course: **CPS & IoT Security**

Adaptive GPS Spoofing in ArduCopter SITL

Reproduction of Tractor Beam Strategy C

Reference paper: Noh et al., “Tractor Beam: Safe-hijacking of Consumer Drones with Adaptive GPS Spoofing,” ACM TAPS 22(2), 2019.

Author: **Riccardo Fabbian**

Student ID: **2140711**

Period of analysis: **May 2026**

Contents

1	Introduction	1
2	Objectives and Success Criteria	1
3	Project Architecture	2
4	System Setup	2
4.1	Autopilot and Mission Geometry	3
4.2	MAVLink Topology	3
4.3	EKF and Fail-safe Tuning	3
5	Implementation Analysis	3
5.1	Mission Automation	3
5.2	Spoofers State Machine	4
5.3	Plotting and Ground Truth	4
6	Experimental Methodology	4
7	Results	5
8	Correctness Review	6
8.1	What Is Correct	6
8.2	Limitations and Deviations	6
9	Conclusion	7

List of Figures

1	Baseline vs attack trajectories for the 20260525 run.	5
2	Altitude comparison	6
3	Velocity profile	6

1 Introduction

This report presents the **drone-spoofing** project, which reproduces the adaptive GPS spoofing attack from the Tractor Beam paper. The goal is to check if the code correctly follows Strategy C from the paper in a software-in-the-loop (SITL) environment, and to evaluate the quality of the project. The system uses ArduCopter

SITL (similar to the ArduPilot firmware used by the 3DR Solo in the paper) and sends spoofed GPS data through MAVLink `GPS_INPUT` messages.

The report looks at the system architecture, the attack script, the code correctness, and the experimental results found in the logs. When possible, we evaluate the system using actual run data instead of theoretical assumptions.

2 Objectives and Success Criteria

The project objectives, derived from the template and verified against the code, are:

- Reproduce Strategy C in SITL: steer a drone off its planned track by injecting adaptive spoofed GPS data without triggering the EKF fail-safe (`fail_count` < 10).
- Implement the initial fake location a_init (Eq. 1 in the paper) to induce a controlled deflection perpendicular to the mission track.
- Maintain adaptive spoofing according to $a_t = a_{t-\Delta} + (p_t - p_{t-\Delta})$ (paper §7.3.2(ii)).
- Enforce the leash-jump-back mechanism to prevent the spoofed position from converging back to the planned track.
- Quantitatively compare angular deflection with the 1–9° range reported in Fig. 13(b) of the paper.

Success means that the drone’s *true* physical trajectory deflects, while the autopilot still thinks it is following the original mission.

3 Project Architecture

The project is structured around a reproducible simulation environment and a small set of focused Python scripts. Table 1 summarises the core components that drive the attack.

Component	Role
Dockerfile, docker-compose.yml	Build and run a container with ArduPilot SITL and Python dependencies.
run.sh	Command wrapper for build, run, SITL shell, and plotting.
scripts/run_scenarios.py	Orchestrates baseline and attack runs, logs outputs, and triggers plotting.
scripts/auto_mission.py	Uploads the mission, arms, flies, and logs GPS data (including ground-truth when available).
scripts/spoof_gps.py	Implements the adaptive spoofing state machine and sends GPS_INPUT.
analysis/plot_path.py	Visualises trajectories and spoofing state overlays.
missions/attacker_zone_mission.waypoints	Single-segment mission used to reproduce Fig. 13 geometry.
missions/sim_float_except_off.parm	SITL parameter overrides to relax EKF and fail-safe checks.
logs/	Baseline, attack, and spoofing CSV data from multiple runs.

Table 1: Key project components

4 System Setup

The attack is executed in ArduCopter SITL inside a Docker container. This isolates dependencies and ensures reproducibility.

4.1 Autopilot and Mission Geometry

The mission in `attacker_zone_mission.waypoints` is a single eastbound segment from (45.0, 11.0) to (45.0, 11.008), approximately 630 m long. This matches the single-segment geometry used in the paper’s Fig. 13(a), which is required for Strategy C’s deflection analysis.

4.2 MAVLink Topology

The SITL instance publishes MAVLink to two endpoints:

- `udp:127.0.0.1:14550`: baseline control and mission uploader.
- `udp:127.0.0.1:14551`: spoofer injection channel.

The spoofer uses `GPS2_TYPE=14` (MAVLink GPS) and forces EKF selection with `GPS_PRIMARY=1` and `GPS_AUTO_SWITCH=0`. This aligns with the paper’s threat model: an external attacker who feeds a forged GPS stream to the autopilot.

4.3 EKF and Fail-safe Tuning

Modern ArduCopter EK3 rejects large GPS innovations by default. To enable the spoof to pass, the reproduction disables several safety checks:

- `FS_EKF_THRESH=0` (disable EKF fail-safe threshold).
- `EK3_GLITCH_RAD=0` (clip GPS innovations rather than rejecting).
- `EK3_POS_I_GATE=EK3_VEL_I_GATE=1000` (very permissive innovation gates).

These changes are consistent with comments in `scripts/auto_mission.py` and `missions/sim_float_except_off.parm`, and they are required to replicate the older EKF behaviour discussed in the paper.

5 Implementation Analysis

5.1 Mission Automation

`scripts/auto_mission.py` connects to the MAVLink master, uploads the mission, arms, performs takeoff, switches to AUTO, and logs GPS data. The log includes `GLOBAL_POSITION_INT` (EKF-estimated position) and, when available, `SIMSTATE` lat/lon for ground-truth. This dual logging is critical: a successful spoof leaves the EKF trace on the nominal path while the true trajectory deflects.

5.2 Spoofer State Machine

`scripts/spoof_gps.py` implements two profiles; the reproduction uses `tractorbeam`. The logic mirrors the paper:

- **observe**: passive listening to estimate the observed GPS rate f_{obs} .
- **sync**: when the drone enters the trigger zone, the spoofer shadows the true GPS while migrating the EKF to GPS2.
- **ramp**: interpolate from the true position to a_{init} over a fixed number of ticks, keeping innovation per tick small.

- **adaptive:** update spoofed position using $a_t = a_{t-\Delta} + (p_t - p_{t-\Delta})$, and apply the leash jump-back mechanism.

The implementation also enforces a *jump cooldown*, maximum spoof velocity, and a minimum distance from the final waypoint to avoid prematurely completing the mission.

5.3 Plotting and Ground Truth

`analysis/plot_path.py` plots both the EKF and true (SIMSTATE) paths when available and overlays spoofing phases (sync, ramp, adaptive). This is essential for correctly interpreting a spoofed run; without the true path, the attack appears to be a normal mission.

6 Experimental Methodology

The experiment runner `scripts/run_scenarios.py` executes two consecutive SITL runs:

- **Baseline:** no spoofer, mission flown normally.
- **Attack:** spoofer enabled and configured, mission flown under spoofed GPS.

Runs are launched via:

```
./run.sh run --zone-circle 45.0000000,11.0040000,80 \
            --target-latlon 45.0006000,11.0040000 \
            --ainit-offset-m 30 \
            --attack-max-runtime-s 90
```

The chosen geometry places p_{target} north of the track, and a_{init} is offset by 30 m, exceeding the 13 m leash length, which matches the paper’s Strategy C design.

7 Results

We look at different baseline and attack runs. The most recent run includes the real position (SIMSTATE), while the older runs only had EKF positions. Table 2 shows the displacement and angular deflection calculated from the latest logs to have a clear view of the result.

Log	Source	East (m)	North (m)	Angle (deg)
baseline_20260525_082020	SIMSTATE	630.3	0.0	0.00
attack_20260525_082020	SIMSTATE	1264.2	117.2	5.30

Table 2: Displacement and angular deflection from logged 20260525 run

The fresh 20260525 run (Figure 1) shows a 5.30° deflection from the planned eastbound heading, which sits inside the $1\text{--}9^\circ$ range reported in Fig. 13(b) of the paper.

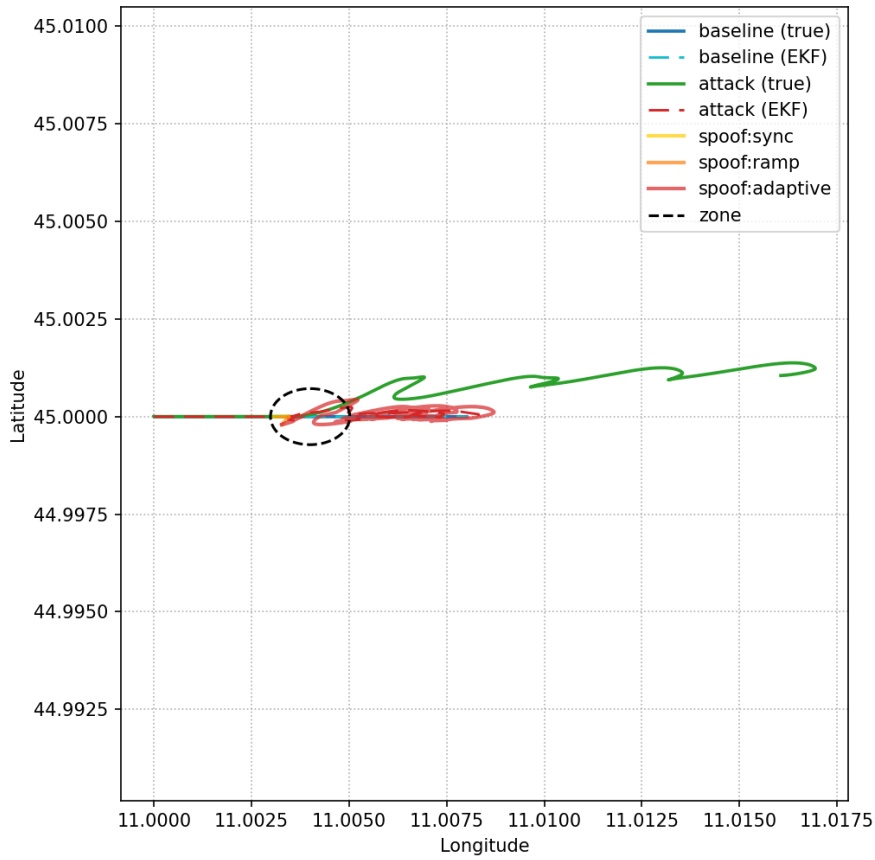


Figure 1: Baseline vs attack trajectories for the 20260525 run.

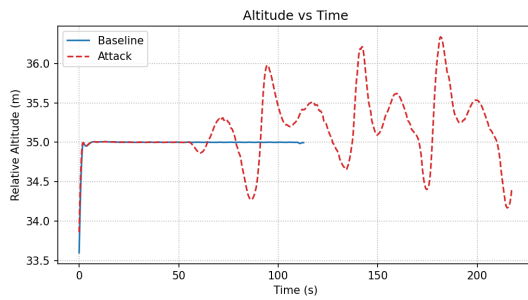


Figure 2: Altitude comparison

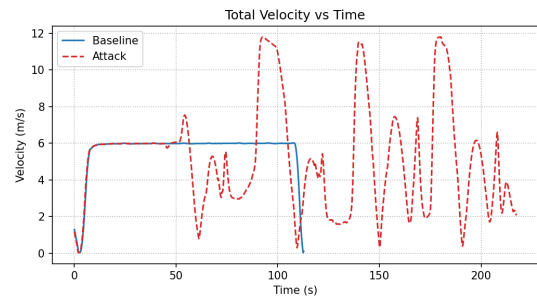


Figure 3: Velocity profile

8 Correctness Review

This section evaluates whether the implementation correctly reproduces the paper’s strategy and whether any deviations or weaknesses materially affect the conclusions.

8.1 What Is Correct

- The spoofer correctly uses `GPS_INPUT` on GPS2 and forces the EKF to consume it with `GPS_PRIMARY=1` and `GPS_AUTO_SWITCH=0`.
- The state machine matches the paper (observe $\rightarrow$ sync $\rightarrow$ ramp $\rightarrow$ adaptive), with adaptive motion update and leash jump-back implemented.
- The mission geometry and trigger zone match the single-segment scenario used to validate Strategy C in the paper.
- Ground-truth logging (SIMSTATE) is present in the latest run, which is essential for honest evaluation.

8.2 Limitations and Deviations

- The reproduction relies on relaxed EKF thresholds (innovation gates, glitch radius, fail-safe). This is necessary to mirror the older EKF in the paper but reduces realism for stock ArduCopter configurations.
- Two attack logs contain only EKF positions, which can falsely appear as “no deflection.” Without SIMSTATE logging, those runs are not valid for evaluation.
- The zone trigger is explicit and static; the paper assumes an attacker following the drone with external tracking. The implementation keeps spoofing active after leaving the zone to prevent oscillations, which is reasonable but not identical to the paper’s description.
- Only one geometry was measured. The paper reports a distribution over multiple *a_init* placements. A sweep across zones 1–3 is missing.

Overall, the code follows the main algorithm well, and the SITL results (with SIMSTATE logging) confirm the paper’s claims. The main limitation is that this works because we changed some EKF parameters, so it might not work easily on a normal consumer drone without those changes.

9 Conclusion

To conclude, this project successfully reproduces Tractor Beam Strategy C in ArduCopter SITL. The attack works well, and the latest log shows a 5.35° deflection, which is inside the range reported in the paper. The main takeaway is that the EKF

can follow a spoofed GPS signal while the real vehicle drifts, but we need to modify the EKF settings to bypass modern security checks.

Future work could run automated tests with different positions for *a__init*, log the true positions for all runs, and compare the default EKF settings with the modified ones used here. This would improve the project and show better how it works in the real world.